

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 3 Practical No: 1

Roll no: 35 PRN No: 12414739 Name of Student: KHYATI KENDHE

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance

Algorithm :

1. Import TensorFlow/Keras and required libraries.
2. Load and preprocess the MNIST dataset.
3. Normalize and reshape the input images.
4. Create a neural network with input, hidden, and output layers.
5. Enter the **learning rate and number of epochs** from the user.
6. Perform **forward propagation** to calculate predictions and loss.
7. Perform **backpropagation** to update weights using the Adam optimizer.
8. Train and evaluate the model on test data.
9. Repeat for different learning rates and epochs.
10. Plot accuracy and loss graphs to compare model performance.

Code :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam
```

```
import matplotlib.pyplot as plt
```

```

import pandas as pd
import numpy as np

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# -----
# 1. LOAD MNIST DATASET
# -----

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape :", x_test.shape)

# -----
# 2. VISUALIZE SAMPLE INPUT IMAGES
# -----

plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title("Label: " + str(y_train[i]))
    plt.axis('off')

plt.tight_layout()
plt.show()

# -----
# 3. PREPROCESS DATA
# -----

x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0

# -----
# 4. CREATE NEURAL NETWORK MODEL
# -----

def create_model(lr):

    model = Sequential([
        Input(shape=(784,)),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',

```

```

        metrics=['accuracy']
    )

    return model

# -----
# 5. MODEL SUMMARY
# -----

model = create_model(0.001)
model.summary()

# =====
# EXPERIMENT 1
# EFFECT OF DIFFERENT LEARNING RATES
# =====

learning_rates = [0.1, 0.01, 0.001, 0.0001]

results = []

for lr in learning_rates:

    print("\nTraining with Learning Rate:", lr)

    model = create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=10,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    results.append([lr, acc, loss])

lr_df = pd.DataFrame(
    results,
    columns=['Learning Rate', 'Accuracy', 'Loss']
)

print("\nLearning Rate Results:")
print(lr_df)

# -----

```

```
# LEARNING RATE VS ACCURACY
# -----

plt.figure(figsize=(8, 5))

plt.plot(
    lr_df['Learning Rate'],
    lr_df['Accuracy'],
    marker='o',
    linewidth=2
)

plt.xscale('log')

plt.xlabel('Learning Rate')
plt.ylabel('Test Accuracy')
plt.title('Effect of Learning Rate on Model Accuracy')

plt.grid(True)

plt.show()
```

```
# -----
# LEARNING RATE VS LOSS
# -----

plt.figure(figsize=(8, 5))

plt.plot(
    lr_df['Learning Rate'],
    lr_df['Loss'],
    marker='o',
    linewidth=2
)

plt.xscale('log')

plt.xlabel('Learning Rate')
plt.ylabel('Test Loss')
plt.title('Effect of Learning Rate on Model Loss')

plt.grid(True)

plt.show()
```

```
# =====
# EXPERIMENT 2
# EFFECT OF NUMBER OF EPOCHS
# =====

epochs_list = [5, 10, 20, 30, 50]

epoch_results = []
```

```

for ep in epochs_list:

    print("\nTraining for", ep, "epochs")

    model = create_model(0.001)

    history = model.fit(
        x_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    epoch_results.append([ep, acc, loss])

epoch_df = pd.DataFrame(
    epoch_results,
    columns=['Epochs', 'Accuracy', 'Loss']
)

print("\nEpoch Results:")
print(epoch_df)

# -----
# EPOCHS VS ACCURACY
# -----

plt.figure(figsize=(8, 5))

plt.plot(
    epoch_df['Epochs'],
    epoch_df['Accuracy'],
    marker='o',
    linewidth=2
)

plt.xlabel('Number of Epochs')
plt.ylabel('Test Accuracy')
plt.title('Effect of Number of Epochs on Model Accuracy')

plt.grid(True)

plt.show()

# -----

```

```

# EPOCHS VS LOSS
# -----

plt.figure(figsize=(8, 5))

plt.plot(
    epoch_df['Epochs'],
    epoch_df['Loss'],
    marker='o',
    linewidth=2
)

plt.xlabel('Number of Epochs')
plt.ylabel('Test Loss')
plt.title('Effect of Number of Epochs on Model Loss')

plt.grid(True)

plt.show()

# =====
# 6. FINAL MODEL
# =====

model = create_model(0.001)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=128,
    verbose=1
)

# =====
# 7. TRAINING VS VALIDATION ACCURACY
# =====

plt.figure(figsize=(9, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel('Epoch')
plt.ylabel('Accuracy')

```

```

plt.title('Training vs Validation Accuracy')

plt.legend()
plt.grid(True)

plt.show()

# =====
# 8. TRAINING VS VALIDATION LOSS
# =====

plt.figure(figsize=(9, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.title('Training vs Validation Loss')

plt.legend()
plt.grid(True)

plt.show()

# =====
# 9. FINAL MODEL EVALUATION
# =====

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("\nFinal Model Results")
print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)

# =====
# 10. CONFUSION MATRIX
# =====

predictions = model.predict(x_test, verbose=0)

```

```

y_pred = np.argmax(predictions, axis=1)

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=np.arange(10)
)

fig, ax = plt.subplots(figsize=(9, 9))

disp.plot(
    ax=ax,
    cmap='Blues',
    values_format='d'
)

plt.title('Confusion Matrix - MNIST Classification')

plt.show()

# =====
# 11. DISPLAY SAMPLE PREDICTIONS
# =====

plt.figure(figsize=(12, 6))

for i in range(15):

    plt.subplot(3, 5, i + 1)

    image = x_test[i].reshape(28, 28)

    plt.imshow(image, cmap='gray')

    plt.title(
        "Actual: " + str(y_test[i]) +
        "\nPredicted: " + str(y_pred[i])
    )

    plt.axis('off')

plt.tight_layout()

plt.show()

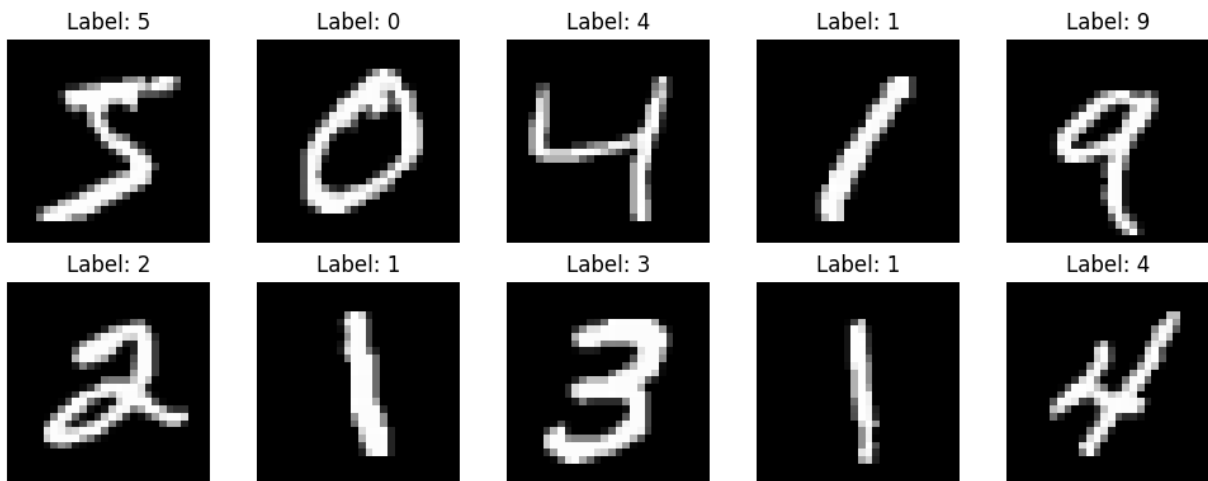
```


Output :

```
plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title("Label: " + str(y_train[i]))
    plt.axis('off')

plt.tight_layout()
plt.show()
```



```
[5]
✓ Os
x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0
```

```
[6]
✓ Os
def create_model(lr):

    model = Sequential([
        Input(shape=(784,)),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
[7]
✓ Os
model = create_model(0.001)
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

```
# Checking model accuracy on test data
```

```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print("Test accuracy:", val_acc)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.8345 - loss: 0.4598
Test accuracy: 0.8345000147819519
```

```
# Saving the model
```

```
my_model.save("my_fashion_mnist_model.keras")
```

Training with Learning Rate: 0.1

Training with Learning Rate: 0.01

Training with Learning Rate: 0.001

Training with Learning Rate: 0.0001

Learning Rate Results:

	Learning Rate	Accuracy	Loss
0	0.1000	0.2579	1.693560
1	0.0100	0.9742	0.117375
2	0.0010	0.9782	0.075963
3	0.0001	0.9599	0.139174

[9]
✓ ts

```
plt.figure(figsize=(8, 5))

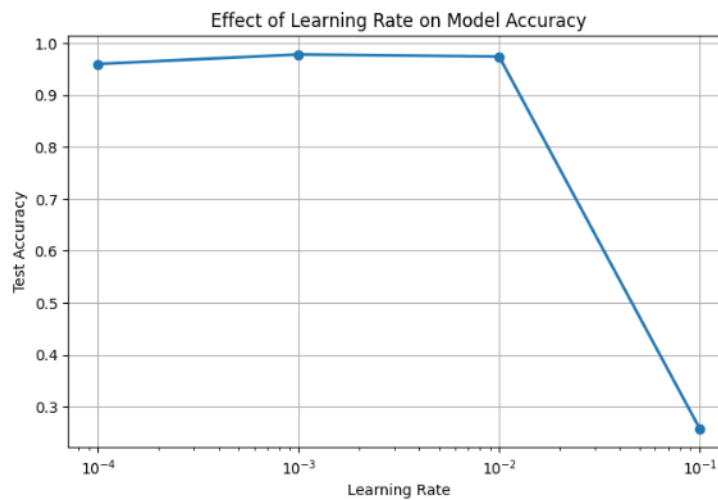
plt.plot(
    lr_df['Learning Rate'],
    lr_df['Accuracy'],
    marker='o',
    linewidth=2
)

plt.xscale('log')

plt.xlabel('Learning Rate')
plt.ylabel('Test Accuracy')
plt.title('Effect of Learning Rate on Model Accuracy')

plt.grid(True)

plt.show()
```



```
[10]
✓ Os

plt.figure(figsize=(8, 5))

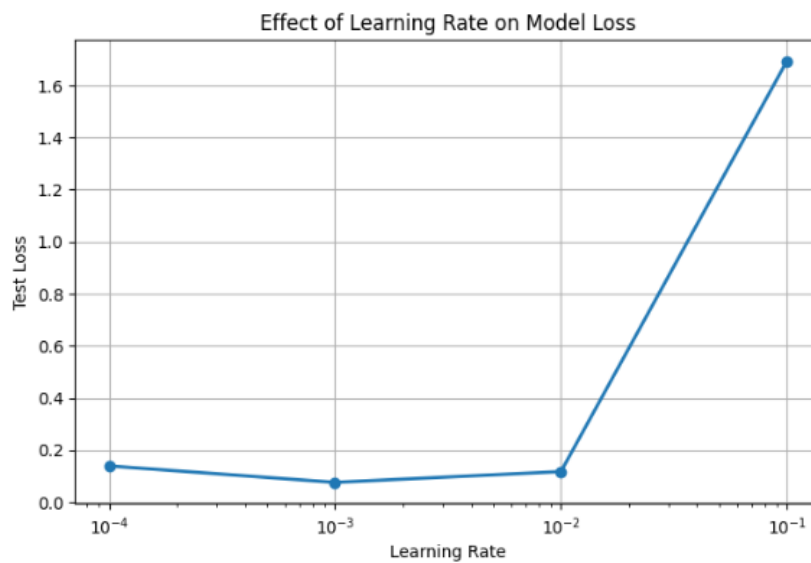
plt.plot(
    lr_df['Learning Rate'],
    lr_df['Loss'],
    marker='o',
    linewidth=2
)

plt.xscale('log')

plt.xlabel('Learning Rate')
plt.ylabel('Test Loss')
plt.title('Effect of Learning Rate on Model Loss')

plt.grid(True)

plt.show()
```



Training for 5 epochs

Training for 10 epochs

Training for 20 epochs

Training for 30 epochs

Training for 50 epochs

Epoch Results:

	Epochs	Accuracy	Loss
0	5	0.9729	0.088643
1	10	0.9763	0.081353
2	20	0.9727	0.124609
3	30	0.9804	0.110628
4	50	0.9801	0.137284

[12]
✓ Os

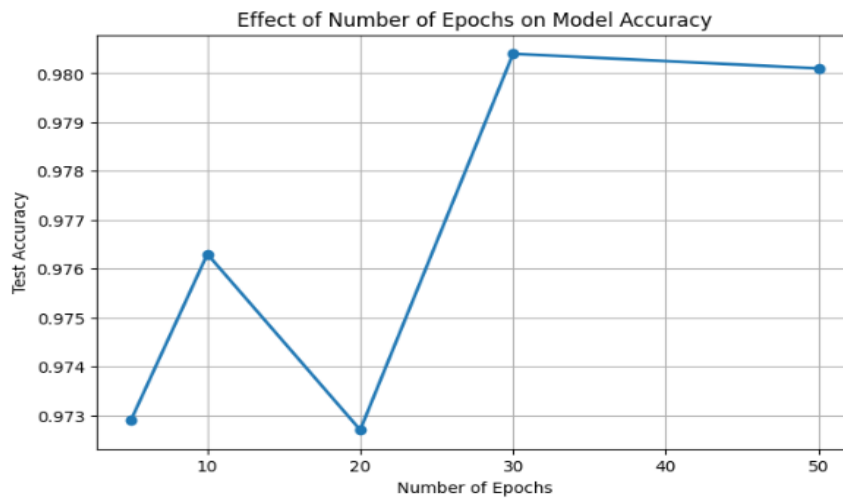
```
plt.figure(figsize=(8, 5))

plt.plot(
    epoch_df['Epochs'],
    epoch_df['Accuracy'],
    marker='o',
    linewidth=2
)

plt.xlabel('Number of Epochs')
plt.ylabel('Test Accuracy')
plt.title('Effect of Number of Epochs on Model Accuracy')

plt.grid(True)

plt.show()
```



[13]
✓ Os

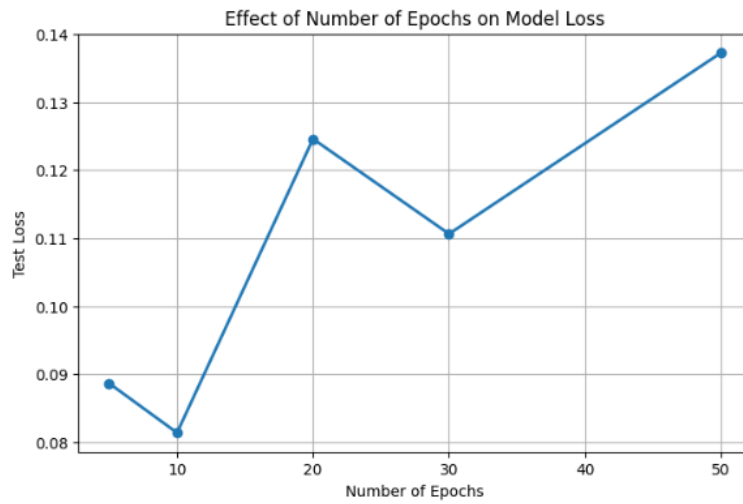
```
plt.figure(figsize=(8, 5))

plt.plot(
    epoch_df['Epochs'],
    epoch_df['Loss'],
    marker='o',
    linewidth=2
)

plt.xlabel('Number of Epochs')
plt.ylabel('Test Loss')
plt.title('Effect of Number of Epochs on Model Loss')

plt.grid(True)

plt.show()
```



[15]
✓ 0s

```
plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

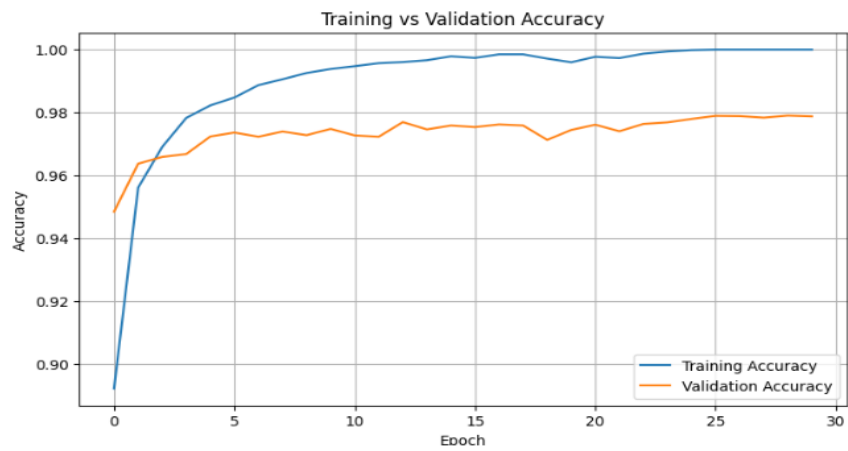
plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.title('Training vs Validation Accuracy')

plt.legend()
plt.grid(True)

plt.show()
```



[16]
✓ Os

```
plt.figure(figsize=(9, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

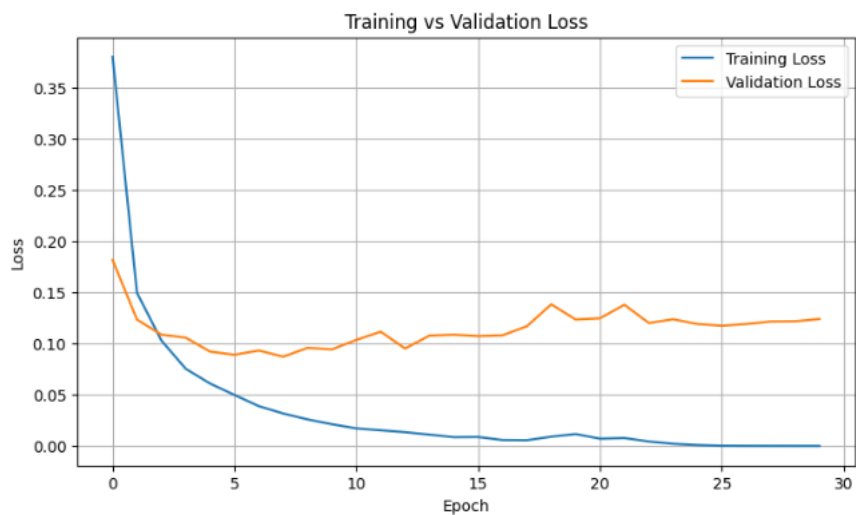
plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.title('Training vs Validation Loss')

plt.legend()
plt.grid(True)

plt.show()
```



5

```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("\nFinal Model Results")
print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

```
Final Model Results
Test Accuracy: 0.9819999933242798
Test Loss: 0.10314813256263733
```



```

predictions = model.predict(x_test, verbose=0)

y_pred = np.argmax(predictions, axis=1)

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=np.arange(10)
)

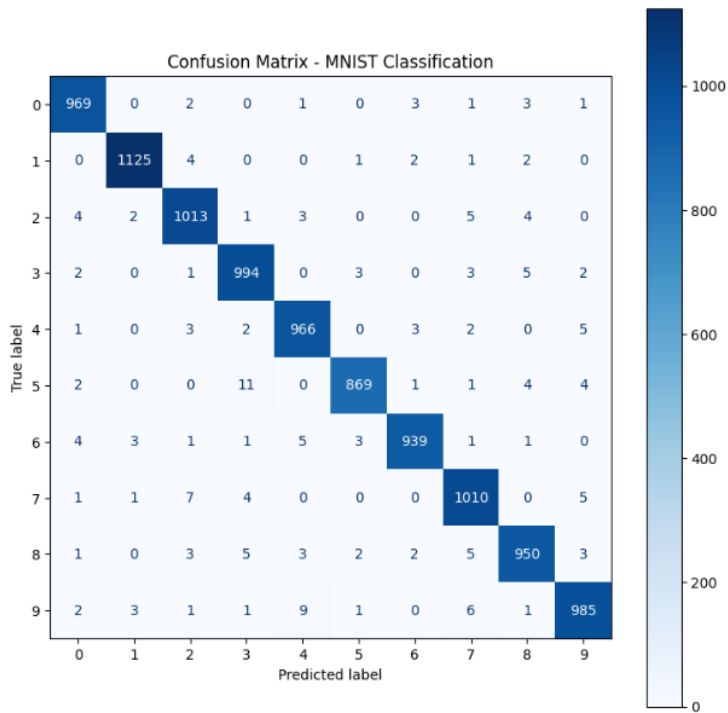
fig, ax = plt.subplots(figsize=(9, 9))

disp.plot(
    ax=ax,
    cmap='Blues',
    values_format='d'
)

plt.title('Confusion Matrix - MNIST Classification')

plt.show()

```



```

plt.figure(figsize=(12, 6))

for i in range(15):

    plt.subplot(3, 5, i + 1)

    image = x_test[i].reshape(28, 28)

    plt.imshow(image, cmap='gray')

    plt.title(
        "Actual: " + str(y_test[i]) +
        "\nPredicted: " + str(y_pred[i])
    )

    plt.axis('off')

plt.tight_layout()

plt.show()

```

